

A REPORT
ON
FLIGHT TICKET MANAGEMENT SYSTEM
BY

K .VENNELA – AP24110010700

D .RADHIKA VARMA - AP24110010648

E .MOHANA KEERTHI – AP2411010667

B. GITANJALI – AP24110010670

K. SINDHURA – AP24110010744



SRM UNIVERSITY, AP

Under the guidance of:

Dr. Balvinder Singh Bondili

Faculty Mentor, Department of Computer Applications

SRM University, AP

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, DR. Balvinder Singh Bondili, for his continuous support, valuable guidance, and encouragement throughout the development of this project. His insightful suggestions and constant motivation helped me to understand the concepts clearly and complete this work successfully.

I am also thankful to all the faculty members of the department for providing me with the necessary knowledge and support required for this project. Their guidance played an important role in the successful completion of my work.

I would like to extend my gratitude to my institution for providing me with the opportunity and resources to carry out this project in a well-organized environment.

I am deeply thankful to my friends and classmates for their cooperation, support, and helpful discussions during the development of this project.

Finally, I would like to express my heartfelt gratitude to my family for their continuous encouragement, understanding, and support, which helped me to complete this project successfully.

CERTIFICATE

This is to certify that the project entitled “FLIGHT MANAGEMENT SYSTEM” has been completed by D.RADHIKA VARMA AP24110010648, E.MOHANA KEERTHI AP24110010667, KANDULA VENNELA AP24110010700, GITANJALI BATHULA – AP24110010670, SINDHURA KAVURI - AP24110010744 as a partial fulfilment of the academic Requirements for the Course CSC209: Database Management System during Semester IV of the Academic Year 2025–2026, in the Department of Computer Science and Engineering at SRM University, AP. This project work has been carried out under the guidance and supervision of the undersigned and is deemed to have satisfactorily met all the requirements prescribed for the course as of April 2026.

Signature of the Course Instructor

DR. Balvinder Singh Bondili

Department of Computer Science and Engineering

SRM University, AP

ABSTRACT

The Flight Database Management System is designed to manage and organize data related to airline operations in an efficient and systematic way. This system helps in maintaining records of flights, passengers, bookings, airports, and payments in a structured format. The main objective of this project is to reduce manual work and improve accuracy in managing airline services.

The system allows users to store details such as flight schedules, passenger information, booking records, and payment details. It also helps in tracking flight routes, managing passenger bookings, and handling payment transactions easily. This makes the overall process faster and more reliable.

The project uses a database management system to ensure data consistency, security, and easy access to information. By implementing this system, airline operations can be handled more efficiently with fewer errors and better record management.

Overall, the Flight Database Management System provides a simple and effective solution for managing airline data and supports better decision-making in daily operations.



TABLE OF CONTENTS

SECTION	PAGE NO
1.IDENTIFICATION OF PROJECT	6
2.PROJECT BACKGROUND	6-7
3. DESCRIPTION OF THE PROJECT	7-8
4. ER DIAGRAM CREATION	8-9
5. DESCRIPTION OF ER DIAGRAM	9-10
6. CONVERSION OF ER DIAGRAM INTO TABLES	10-12
7. DESCRIPTION OF THE TABLES	13-16
8. NORMALISATION OF TABLES UPTO 3NF	16-18
9. CREATION OF DATA IN THE TABLES	19-20
10. SQL QUERIES	20-24
11. CREATION OF VIEWS	25-27
12. CONCLUSION	27
13.REFERENCES	27

PROJECT REPORT

1. Identification of Project related to DBMS

Flight Ticket Management System

FIELD	DETAILS
Project Title	Flight Ticket Management System
Course	Database Management System
Course Code	CSE-209
Semester	4
Department	Computer Science and Engineering
Team Members	D. Radhika varma, E. Mohana Keerthi, B. Gitanjali ,K. Sindhura

2. Project Background

Air transportation is one of the fastest-growing sectors in the world. Managing flight bookings manually can lead to errors such as incorrect reservations, data loss, and inefficiency in handling passenger details.

With increasing demand for online ticket booking, there is a need for an automated system that can manage flight schedules, passenger records, bookings, and payments efficiently.

The **Flight Ticket Management System** is designed to overcome these problems by using a database system that ensures accuracy, fast data retrieval, and secure data storage.

Motive

- Reduce manual errors in booking system
- Store passenger and flight data efficiently
- Provide fast booking and cancellation
- Maintain payment records securely
- Improve overall management of airline operations

3.Description of the Project

The Flight Ticket Management System is a relational database system designed to manage airline ticket booking operations.

Core Modules

3.1 Passenger Management

Stores passenger details such as ID, name, age, and phone number.

3.2 Flight Management

Maintains information about flights including source, destination, and departure time.

3.3 Booking Management

Handles ticket booking by linking passengers with flights.

3.4 Ticket Management

Stores seat number and class details for each booking.

3.5 Payment Management

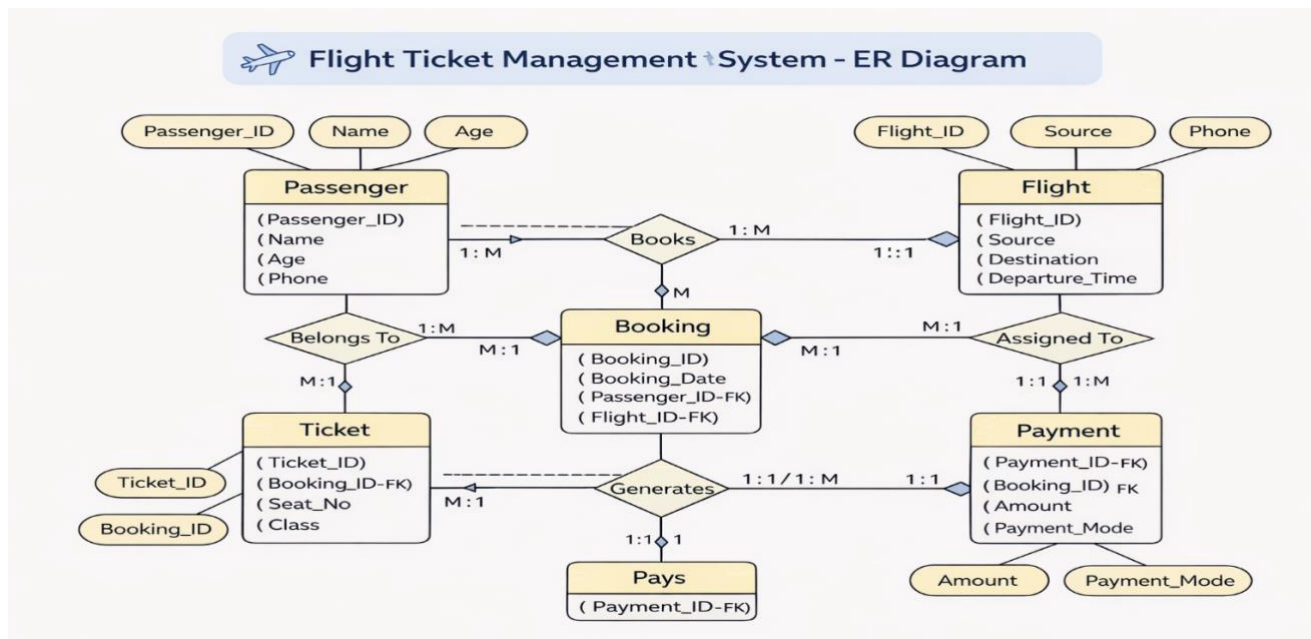
Maintains payment details like amount and payment mode.

Key Features

- Easy ticket booking and cancellation
- Secure data storage
- Fast retrieval of passenger and flight details
- Accurate billing system
- Data consistency using relational model

4.ER Diagram Creation

The ER diagram represents the structure of the system using entities, attributes, and relationships.





Entities Identified

- Passenger
- Flight
- Booking
- Ticket
- Payment

5.Description of ER Diagram

Entities and Attributes

Entity	Attributes	Primary Key	Foreign Key
Passenger	Passenger_ID, Name, Age, Phone	Passenger_ID	_____
Flight	Flight_ID, Source, Destination, Departure_Time	Flight_ID	_____
Booking	Booking_ID, Booking_Date, Passenger_ID, Flight_ID	Booking_ID	Passenger_ID, Flight_ID
Ticket	Ticket_ID, Booking_ID, Seat_No, Class	Ticket_ID	Booking_ID
Payment	Payment_ID, Booking_ID, Amount, Payment_Mode	Payment_ID	Booking_ID

Relationships

Relationship	Entities	Type
Books	Passenger → Booking	1 : M
Assigned To	Flight → Booking	1 : M
Generates	Booking → Ticket	1 : 1
Pays	Booking → Payment	1 : 1

6.Conversion of ER Diagram into Tables

The ER diagram of the Flight Database Management System is converted into relational tables using standard mapping rules. Each entity in the ER diagram is transformed into a table, and relationships are represented using foreign keys.

The many-to-many relationship between Booking and Flight is resolved using a junction table called Ticket

Mapping Rules Applied

- 1.Each entity in the ER diagram is converted into a separate table.
- 2.All attributes of the entity become columns in the table.
- 3.Primary keys are defined using the PRIMARY KEY constraint.
4. one-to-many relationships are implemented using foreign keys in the “many” side table.
- 5.Many-to-many relationships are resolved using a junction table with a composite primary key.

Resulting Tables

Passenger Table

```
mysql> CREATE TABLE Passenger (  
->     Passenger_ID INT PRIMARY KEY,  
->     Name VARCHAR(50),  
->     Age INT,  
->     Phone VARCHAR(15)  
-> );
```

Flight Table

```
mysql> CREATE TABLE Flight (  
->     Flight_ID VARCHAR(10) PRIMARY KEY,  
->     Source VARCHAR(50),  
->     Destination VARCHAR(50),  
->     Departure_Time VARCHAR(20)  
-> );
```

Booking Table

```
mysql> CREATE TABLE Booking (  
->     Booking_ID INT PRIMARY KEY,  
->     Passenger_ID INT,  
->     Flight_ID VARCHAR(10),  
->     Booking_Date DATE,  
->     FOREIGN KEY (Passenger_ID) REFERENCES Passenger(Passenger_ID),  
->     FOREIGN KEY (Flight_ID) REFERENCES Flight(Flight_ID)  
-> );  
Query OK, 0 rows affected (0.05 sec)
```

Ticket Table

```
mysql>  
mysql> CREATE TABLE Ticket (  
->     Ticket_ID INT PRIMARY KEY,  
->     Booking_ID INT,  
->     Seat_No VARCHAR(10),  
->     Class VARCHAR(20),  
->     FOREIGN KEY (Booking_ID) REFERENCES Booking(Booking_ID)  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

Payment Table

```
mysql>  
mysql> CREATE TABLE Payment (  
->     Payment_ID INT PRIMARY KEY,  
->     Booking_ID INT,  
->     Amount DECIMAL(10,2),  
->     Payment_Mode VARCHAR(20),  
->     FOREIGN KEY (Booking_ID) REFERENCES Booking(Booking_ID)  
-> );
```

7.Description of Tables

This table stores information about all airports. Each airport has a unique ID.



Passenger Table

Stores passenger details.

Column	Data Type	Constraint	Description
Passenger_ID	INT	PRIMARY KEY	Unique passenger ID
Name	VARCHAR(50)	NOT NULL	Passenger name
Age	INT	NOT NULL	Passenger age
Phone	VARCHAR(15)	NOT NULL	Contact number

Sample Data

```
mysql> SELECT * FROM Passenger;
```

```
+-----+-----+-----+-----+
| Passenger_ID | Name | Age | Phone |
+-----+-----+-----+-----+
|          1 | Ravi | 25 | 9876543210 |
|          2 | Sita | 30 | 9123456780 |
|          3 | Rahul | 28 | 9012345678 |
+-----+-----+-----+-----+
3 rows in set (0.01 sec)
```

Flight Table

Stores flight information.



Column	Data Type	Constraint	Description
Flight_ID	VARCHAR(10)	PRIMARY KEY	Unique flight ID
Source	VARCHAR(50)	NOT NULL	Starting location
Destination	VARCHAR(50)	NOT NULL	Ending location
Departure_Time	DATETIME	NOT NULL	Flight departure time
Airport_ID	INT	FOREIGN KEY	References Airport

Sample Data

```
mysql> SELECT * FROM Flight;
+-----+-----+-----+-----+
| Flight_ID | Source   | Destination | Departure_Time |
+-----+-----+-----+-----+
| F101      | Delhi    | Mumbai      | 10:00 AM       |
| F102      | Hyderabad | Chennai     | 2:00 PM        |
| F103      | Bangalore | Kolkata     | 6:00 PM        |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Booking Table

Links passengers and flights.

Column	Data Type	Constraint	Description
Booking_ID	INT	PRIMARY KEY	Unique booking ID
Date	DATE	NOT NULL	Booking date
Passenger_ID	INT	FOREIGN KEY	References Passenger



Sample Data

```
mysql> SELECT * FROM Booking;
+-----+-----+-----+-----+
| Booking_ID | Passenger_ID | Flight_ID | Booking_Date |
+-----+-----+-----+-----+
| 101 | 1 | F101 | 2026-04-20 |
| 102 | 2 | F102 | 2026-04-21 |
| 103 | 3 | F103 | 2026-04-22 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Ticket Table

Stores seat and class details.

Column	Data Type	Constraint	Description
Booking_ID	INT	PK + FK	References Booking
Flight_ID	VARCHAR(10)	PK + FK	References Flight

Sample Data

```
mysql> SELECT * FROM Ticket;
+-----+-----+-----+-----+
| Ticket_ID | Booking_ID | Seat_No | Class |
+-----+-----+-----+-----+
| 201 | 101 | A1 | Economy |
| 202 | 102 | B2 | Business |
| 203 | 103 | C3 | Economy |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Payment Table

Stores transaction details.

Column	Data Type	Constraint	Description
Payment_ID	INT	PRIMARY KEY	Unique payment ID
Booking_ID	INT	FOREIGN KEY	References Booking
Amount	INT	NOT NULL	Payment amount
Payment_Date	DATE	NOT NULL	Date of payment
Payment_Mode	VARCHAR(20)	NOT NULL	Mode (UPI/Card/Cash)

Sample Data

```
mysql> SELECT * FROM Payment;
```

```
+-----+-----+-----+-----+
| Payment_ID | Booking_ID | Amount | Payment_Mode |
+-----+-----+-----+-----+
|          301 |          101 | 5000.00 | UPI           |
|          302 |          102 | 8000.00 | Card          |
|          303 |          103 | 6500.00 | Cash          |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

8.Normalization of Tables up to 3NF

Normalization is the process of organizing data in a database to reduce duplication and improve accuracy. All tables in the Flight Management System are normalized up to Third Normal Form (3NF).

First Normal Form (1NF)

A table is in 1NF if:

- All values are atomic (single value)
- No repeating columns
- Each row is unique

Application:

- All tables have simple values (no multiple values in one column)
- Each table has a primary key

Condition	Application in Flight DB
Atomic values (no multiple values in one column)	All columns store single values
No repeating groups	No repeated columns in any table
Each row is unique	All tables have primary keys

Second Normal Form (2NF)

A table is in 2NF if:

- It is already in 1NF
- No partial dependency (data depends on full primary key)

Application (Ticket Table):

- Primary key = (Booking_ID, Flight_ID)
- No column depends only on one part



Condition	Application in Flight DB
Must be in 1NF	Already satisfied
No partial dependency	In Ticket table, data depends on full key (Booking_ID + Flight_ID)

Third Normal Form (3NF)

A table is in 3NF if:

- It is in 2NF
- No transitive dependency (non-key depends only on primary key)

Application:

- No unnecessary columns
- All data depends only on primary key

Table	Primary Key	Non-Key Attributes	Transitive Dependency
Airport	Airport_ID	Name, Location	No
Flight	Flight_ID	Source, Destination, Departure_Time, Airport_ID	No
Passenger	Passenger_ID	Name, Age, Phone	No
Booking	Booking_ID	Date, Passenger_ID	No
Ticket	(Booking_ID, Flight_ID)	—	No
Payment	Payment_ID	Booking_ID, Amount, Payment_Date, Payment_Mode	No

9. Creation of Data in the Tables

In this section, sample data is inserted into all tables of the Flight Management System. This data helps to test the database and understand how the system works.

Passenger Data

```
INSERT INTO Passenger VALUES (1, 'Ravi', 25, '9876543210');  
INSERT INTO Passenger VALUES (2, 'Sita', 30, '9123456780');  
INSERT INTO Passenger VALUES (3, 'Rahul', 28, '9012345678');
```

Flight Data

```
INSERT INTO Flight VALUES ('F101', 'Delhi', 'Mumbai', '10:00 AM');  
INSERT INTO Flight VALUES ('F102', 'Hyderabad', 'Chennai', '2:00 PM');  
INSERT INTO Flight VALUES ('F103', 'Bangalore', 'Kolkata', '6:00 PM');
```

Booking Data

```
INSERT INTO Booking VALUES (101, 1, 'F101', '2026-04-20');  
INSERT INTO Booking VALUES (102, 2, 'F102', '2026-04-21');  
INSERT INTO Booking VALUES (103, 3, 'F103', '2026-04-22');
```

Ticket Data

INSERT INTO Ticket VALUES (201, 101, 'A1', 'Economy');

INSERT INTO Ticket VALUES (202, 102, 'B2', 'Business');

INSERT INTO Ticket VALUES (203, 103, 'C3', 'Economy');

Payment Data

INSERT INTO Payment VALUES (301, 101, 5000, 'UPI');

INSERT INTO Payment VALUES (302, 102, 8000, 'Card');

INSERT INTO Payment VALUES (303, 103, 6500, 'Cash');

10. SQL Queries – Joins, Aggregate Functions, and Subqueries

In this section, different SQL queries are used to retrieve data from the Flight Management System. These queries help in understanding how data is connected and used.

10.1 JOIN Queries

Query 1: (Passenger + Flight)



```
SELECT P.Name, F.Source, F.Destination
```

```
FROM Passenger P
```

```
JOIN Booking B ON P.Passenger_ID = B.Passenger_ID
```

```
JOIN Flight F ON B.Flight_ID = F.Flight_ID;
```

Expected Output:

Name	Source	Destination
Ravi	Delhi	Mumbai
Sita	Hyderabad	Chennai
Rahul	Bangalore	Kolkata

3 rows in set (0.04 sec)

Query 2: Total Revenue

```
SELECT SUM(Amount) AS Total_Revenue FROM Payment;
```

Total_Revenue
19500.00

1 row in set (0.01 sec)

Query 3: Count Bookings

```
SELECT COUNT(*) FROM Booking;
```

COUNT(*)
3

10.2 Aggregate Function Queries

SELECT COUNT(*) AS Total_Bookings FROM Booking;

SELECT SUM(Amount) AS Total_Revenue FROM Payment;

SELECT AVG(Amount) AS Average_Payment FROM Payment;

Expected output:

```
mysql> SELECT COUNT(*) AS Total_Bookings FROM Booking;
+-----+
| Total_Bookings |
+-----+
|          3 |
+-----+
1 row in set (0.01 sec)

mysql>
mysql> SELECT SUM(Amount) AS Total_Revenue FROM Payment;
+-----+
| Total_Revenue |
+-----+
|    19500.00 |
+-----+
1 row in set (0.00 sec)

mysql>
mysql> SELECT AVG(Amount) AS Average_Payment FROM Payment;
+-----+
| Average_Payment |
+-----+
|    6500.000000 |
+-----+
1 row in set (0.00 sec)
```



Query 4: GROUP BY

```
SELECT Payment_Mode, COUNT(*)  
FROM Payment  
GROUP BY Payment_Mode;
```

Expected Output:

Payment_Mode	COUNT(*)
UPI	1
Card	1
Cash	1

Query 5: ORDER BY

```
SELECT * FROM Passenger ORDER BY Age DESC;
```

Passenger_ID	Name	Age	Phone
2	Sita	30	9123456780
3	Rahul	28	9012345678
1	Ravi	25	9876543210



10.3 Subquery

```
SELECT Name
FROM Passenger
WHERE Passenger_ID IN (
    SELECT Passenger_ID FROM Booking
);
```

Expected Output:

Name
Ravi
Sita
Rahul

Query 6: UPDATE

```
UPDATE Passenger
SET Phone = '9999999999'
WHERE Passenger_ID = 1;
```

Query 7: DELETE

```
DELETE FROM Ticket
WHERE Ticket_ID = 202;
```


11. Creation of Views Using the Tables (Simple)

A **View** in a database is a virtual table created using a SQL query. It does not store data physically but shows data from one or more tables. Views are useful for simplifying complex queries and improving data security.

In the Flight Ticket Management System, views are created to easily access important information.

View 1: Booking Details

This view shows passenger name along with flight details.

```
CREATE VIEW Booking_Details AS  
SELECT P.Name, F.Source, F.Destination  
FROM Passenger P  
JOIN Booking B ON P.Passenger_ID = B.Passenger_ID  
JOIN Flight F ON B.Flight_ID = F.Flight_ID;
```

Usage:

```
SELECT * FROM Booking_Details;
```

View 2: Payment Details

This view displays payment information for each booking.

```
CREATE VIEW Payment_Details AS
```



```
SELECT Booking_ID, Amount, Payment_Mode  
FROM Payment;
```

Usage:

```
SELECT * FROM Payment_Details;
```

View 3: Passenger Details

This view shows basic passenger information.

```
CREATE VIEW Passenger_Info AS  
SELECT Name, Age, Phone  
FROM Passenger;
```

Usage:

```
SELECT * FROM Passenger_Info;
```

View 4: Flight Details

This view shows flight information.

```
CREATE VIEW Flight_Info AS  
SELECT Source, Destination, Departure_Time  
FROM Flight;
```

Usage:

```
SELECT * FROM Flight_Info;
```

12. Conclusion

The Flight Ticket Management System successfully demonstrates how DBMS concepts can be applied to real-world applications. It provides efficient management of passenger data, flight schedules, bookings, and payments.

The system ensures:

- Data accuracy
- Reduced redundancy
- Fast access to information

13. References

- DBMS Textbook
- Lecture Notes
- Online Tutorials